```
import pandas as pd
import numpy as np
```

```
df = pd.read_csv("crop_prices.csv")
print(df.head())
```

```
            state      district             market commodity  variety  grade  \
0  Madhya Pradesh       Badwani  Badwani(F&V) APMC   Pumpkin  Pumpkin  Local
1  Madhya Pradesh      Shivpuri   Karera(F&V) APMC    Potato   Potato  Local
2  Madhya Pradesh      Shivpuri   Karera(F&V) APMC     Onion    Onion  Local
3  Madhya Pradesh      Shivpuri   Karera(F&V) APMC   Raddish  Raddish  Local
4   Uttar Pradesh  Bulandshahar  Anoop Shahar APMC   Pumpkin    Other    FAQ

  arrival_date  min_price  max_price  modal_price
0   01/03/2026     1200.0     1200.0       1200.0
1   01/03/2026      500.0     1000.0        600.0
2   01/03/2026     1200.0     1800.0       1500.0
3   01/03/2026      600.0     1000.0        800.0
4   01/03/2026     1000.0     1400.0       1200.0
```

```
df['target'] = df['modal_price'].apply(lambda x: 1 if x > 1000 else 0)
```

```
threshold = df['modal_price'].median()

df['target'] = df['modal_price'].apply(lambda x: 1 if x > threshold else 0)
```

```
print(np.bincount(df['target']))
```

```
[4306 4272]
```

```
X = df[['min_price', 'max_price']].values
y = df['target'].values
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
import random

combined = list(zip(X, y))
random.shuffle(combined)

X, y = zip(*combined)
X = np.array(X)
y = np.array(y)
```

```
class Perceptron:
    def __init__(self, lr=0.01, epochs=20):
        self.lr = lr
        self.epochs = epochs
```

```
    def activation(self, z):
        return 1 if z >= 0 else 0

    def fit(self, X, y):
        self.weights = np.zeros(X.shape[1])
        self.bias = 0

        for _ in range(self.epochs):
            for i in range(len(X)):
                z = np.dot(X[i], self.weights) + self.bias
                y_pred = self.activation(z)

                error = y[i] - y_pred

                self.weights += self.lr * error * X[i]
                self.bias += self.lr * error

    def predict(self, X):
        result = []
        for x in X:
            z = np.dot(x, self.weights) + self.bias
            result.append(self.activation(z))
        return result
```

```
model = Perceptron(lr=0.01, epochs=20)
model.fit(X, y)
```

```
print("Weights:", model.weights)
print("Bias:", model.bias)
```

```
Weights: [0.06952182 0.08310385]
Bias: 0.03
```

```
pred = model.predict(X[:10])
print("Predictions:", pred)
print("Actual:", y[:10])
```

```
Predictions: [1, 0, 0, 0, 0, 0, 1, 0, 1, 1]
Actual: [1 0 0 0 0 0 1 0 1 1]
```

```
y_pred = model.predict(X)

accuracy = np.mean(y_pred == y)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9989508043833061
```

BackPropagtion

```
from sklearn.neural_network import MLPClassifier
import numpy as np

# model (backprop handled internally)
model = MLPClassifier(
```

```
    hidden_layer_sizes=(4, 4),   # 2 hidden layers
    activation='relu',
    solver='adam',
    learning_rate_init=0.01,
    max_iter=500
)

# train
model.fit(X, y)

# predict
y_pred = model.predict(X)

# accuracy
accuracy = np.mean(y_pred == y)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.9993005362555374
```

Feedback Neural Network

```
class NeuralNetwork:
    def __init__(self, input_size, hidden_size, lr=0.01, epochs=100):
        self.lr = lr
        self.epochs = epochs

        # weights initialization
        self.W1 = np.random.randn(input_size, hidden_size)
        self.b1 = np.zeros(hidden_size)

        self.W2 = np.random.randn(hidden_size, 1)
        self.b2 = 0

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def sigmoid_derivative(self, x):
        return x * (1 - x)

    def fit(self, X, y):
        y = y.reshape(-1, 1)

        for epoch in range(self.epochs):
            # ---- Forward Pass ----
            z1 = np.dot(X, self.W1) + self.b1
            a1 = self.sigmoid(z1)

            z2 = np.dot(a1, self.W2) + self.b2
            a2 = self.sigmoid(z2)

            # ---- Error (Feedback) ----
            error = y - a2

            # ---- Backpropagation ----
            d_output = error * self.sigmoid_derivative(a2)
```

```
            error_hidden = d_output.dot(self.W2.T)
            d_hidden = error_hidden * self.sigmoid_derivative(a1)

            # ---- Update weights ----
            self.W2 += self.lr * a1.T.dot(d_output)
            self.b2 += self.lr * np.sum(d_output)

            self.W1 += self.lr * X.T.dot(d_hidden)
            self.b1 += self.lr * np.sum(d_hidden, axis=0)

            if epoch % 10 == 0:
                loss = np.mean(error**2)
                print(f"Epoch {epoch}, Loss: {loss}")

    def predict(self, X):
        z1 = np.dot(X, self.W1) + self.b1
        a1 = self.sigmoid(z1)

        z2 = np.dot(a1, self.W2) + self.b2
        a2 = self.sigmoid(z2)

        return (a2 > 0.5).astype(int).flatten()
```

```
model = NeuralNetwork(input_size=2, hidden_size=4, lr=0.01, epochs=100)
model.fit(X, y)

y_pred = model.predict(X)

accuracy = np.mean(y_pred == y)
print("Accuracy:", accuracy)
```

```
Epoch 0, Loss: 0.19943380221717097
Epoch 10, Loss: 0.04481934065704988
Epoch 20, Loss: 0.062370103765011585
Epoch 30, Loss: 0.03716725380517138
Epoch 40, Loss: 0.025658906099041673
Epoch 50, Loss: 0.019099577786582608
Epoch 60, Loss: 0.015893362829118893
Epoch 70, Loss: 0.014166653574662685
Epoch 80, Loss: 0.013027553810227698
Epoch 90, Loss: 0.012141131692042069
Accuracy: 0.9910235486127302
```

FeedForword

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(
    hidden_layer_sizes=(4, 4),
    activation='relu',
    max_iter=500
)

model.fit(X, y)

y_pred = model.predict(X)
print("Accuracy:", np.mean(y_pred == y))
```

```
print("Accuracy:", np.mean(y_pred == y))
```

```
Accuracy: 0.9993005362555374
```